

Cloud-Native Microservices: Architecture and Scaling Strategies

Abstract

Organizations are migrating monolithic applications to cloud-native microservice architectures to improve scalability, deployment velocity, and fault isolation. This paper surveys container orchestration, service mesh, and autoscaling techniques used to operate microservices reliably in production cloud environments.

1. Methodology

We containerize each service using Docker and orchestrate deployment using Kubernetes, which manages scheduling, self-healing, and horizontal pod autoscaling based on CPU and custom metrics. A service mesh handles inter-service traffic routing, retries, and observability. Infrastructure is provisioned declaratively using infrastructure-as-code, enabling reproducible environments across staging and production.

2. Advantages and Limitations

The primary advantage of this architecture is elastic scalability: individual services can be scaled independently based on demand, avoiding the need to scale an entire monolith. Fault isolation is also improved, since a failure in one service does not necessarily cascade to others. A limitation is increased operational complexity: distributed tracing, service discovery, and network policy management require dedicated tooling and expertise that a monolithic deployment does not need.

3. Conclusion

We conclude that cloud-native microservices provide substantial scalability and deployment benefits for organizations with sufficient operational maturity, but that the added complexity is a meaningful cost that should be weighed against the specific scaling requirements of the application.